

VisionEPI — Documento técnico consolidado (Fases 1 a 4)

Challenge 2026 — Innovation Challenge CUP — Parceria FIAP x SPI

Engenharia da Computação, 3º ano.

Equipe

Papel	Nome
Aluno	Lucas Baraldi Rodrigues
Aluno	Lucas Zolla Assis
Aluno	Pedro Costa Belisário
Aluno	Vitor Pantarotto de Brito
Professor	Fabio Henrique Pimentel
Mentor SPI	Fernando Marcolina
Mentor SPI	Wendel de Almeida Passos

Cronograma

Marco	Data
Início do projeto	16/04/2026
Entrega Sprint 1	04/05/2026
Entrega Sprint 2	22/06/2026
Entrega Sprint 3	24/08/2026
Entrega Sprint 4 e banca final	23/09/2026

Ferramenta de planejamento: Trello, com colunas por fase (Backlog -> Fase 1 -> Fase 2 -> Fase 3 -> Fase 4 -> Concluído).

Feedback recebido da SPI: "Boa evolução do projeto, boa qualidade de detecção" — incorporado nas fases seguintes mantendo o padrão de qualidade de detecção elogiado e evoluindo a robustez (ensemble de dois modelos, GPU, rastreamento de pessoa) e o escopo (pose, notificação inteligente, validação em câmeras reais).

1. Resumo executivo

O VisionEPI é um sistema de monitoramento de segurança do trabalho por visão computacional: detecta o uso de equipamentos de proteção individual (EPIs) por câmera, em tempo real, classifica postura e queda, gera alertas com

política de severidade explícita, e controla o acesso físico por um modo portaria que só libera a entrada de quem está com o EPI exigido — **na dúvida, nega**.

O sistema roda hoje contra **câmeras reais de uma planta industrial parceira** (RTSP, protocolo Dahua) e contra webcam, com um dashboard web para três perfis de acesso (operador, técnico, supervisor), cada um vendo só o que precisa.

2. Fase 1 — Detecção de EPIs / EPCs

2.1 O que foi construído

- **Ensemble de dois modelos YOLOv8**, rodando no mesmo quadro: o **Vyra** ([Hexmon/vyra-yolo-ppe-detection](#)), pesos prontos, licença CC BY 4.0) e um modelo **treinado pela equipe** no dataset **SH17** (8.099 imagens, 75.994 instâncias, licença CC BY-NC-SA 4.0 — uso acadêmico), fundidos por IoU.
- **7 classes de EPI** detectadas: capacete, colete, luvas, óculos, máscara, calçado de segurança e **protetor auricular** (adicionado nesta fase final, ver §5).
- **Três estados por item, não dois**: "em uso correto" (ok), "ausente" (missing) e **"em uso incorreto"** (incorrect — o item existe sobre a pessoa, mas fora da posição esperada, ex.: capacete na mão em vez de na cabeça). Implementado por geometria: cada EPI tem uma faixa vertical esperada no corpo da pessoa; dentro da faixa é "ok", fora da faixa mas ainda sobre o corpo é "incorreto", fora do corpo é "ausente".
- **Associação geométrica exclusiva EPI->pessoa**: pontuação por contenção da caixa + proximidade da faixa vertical ideal, resolvendo o caso de duas pessoas sobrepostas disputando o mesmo capacete.
- **Rastreador de pessoas** (IoU + distância entre centros como plano B), mantendo o mesmo número de identificação mesmo quando a pessoa muda de postura.

2.2 Métricas

Validação por classe do modelo SH17 (`model.val()`), Ultralytics, imgsz 640, GPU, conjunto de **teste** — 810 imagens, nunca usadas no treino nem na validação durante o treino):

EPI nosso	Classe SH17	Precisão	Revocação	F1	mAP@0.5	mAP@0.5:0.95
Capacete	helmet	0,810	0,512	0,628	0,669	0,441
Colete	safety-vest	0,810	0,433	0,564	0,490	0,323
Luvas	gloves	0,678	0,527	0,593	0,570	0,351
Óculos	glasses	0,712	0,678	0,694	0,696	0,411
Máscara	face-mask	0,837	0,588	0,691	0,696	0,472
Calçado	shoes	0,745	0,570	0,646	0,615	0,332
Protetor auricular	ear-acessorio	0,654	0,235	0,346	0,244	0,151
Agregado (17 classes do SH17)	—	0,733	0,588	—	0,621	0,414

Achado honesto: o **protetor auricular é o item mais fraco de longe** — só 17 instâncias no conjunto de teste, a classe mais rara do dataset inteiro, e a revocação cai para 0,235. Os demais itens ficam na faixa de 0,49 a 0,70 de mAP@0.5, abaixo da meta de 0,75 do guia, com destaque positivo para óculos e máscara.

O **ensemble completo** (Vyra + SH17, fundidos por IoU, config idêntica à de produção) também foi medido contra o mesmo teste: saiu **pior em mAP@0.5** que o SH17 isolado na maioria das classes (só o protetor auricular melhorou), mas com **precisão bem maior** em óculos (0,71->0,81) e máscara (0,84->0,94). Não é regressão — o gabarito de teste é do SH17, não do Vyra (treinado noutro dataset, caixas calibradas diferente), então o número mede mal a contribuição real do Vyra. Detalhe completo e a tabela em [FASE1-DETECCA0-DE-EPIS.md](#) §3.1.

O **detector de pessoa** (pré-requisito de EPI e pose — YOLOv8n/COCO dedicado, já que o Vyra sozinho não cobre bem várias pessoas na cena) saiu com **mAP@0.5 = 0,740**, P=0,759, R=0,698, F1=0,727 — a melhor métrica de todo o sistema, e esperado: "pessoa" é o caso de uso mais maduro do YOLO. Detalhe em [FASE1-DETECCA0-DE-EPIS.md](#) §3.2.

[ATENCAO] **Limitação declarada:** as métricas acima são contra o conjunto de teste do SH17 — não contra imagens da planta real, e não é o terreno de teste nativo do Vyra.

- **FPS:** ~19,5 quadros/segundo com o pipeline completo numa câmera, GPU (RTX). Em múltiplas câmeras simultâneas, cai — ver §5, achado da validação real.
- **Latência de inferência:** dentro do orçamento de quadro a 12 FPS de captura.

2.3 Inovações da fase

- Ensemble de dois modelos em vez de um só, para cobrir classes que o modelo principal detecta mal.
- Distinção correto/incorrecto/ausente (não só presença binária).
- Regra situacional: EPI exigido varia por câmera (ver Fase 3, modo portaria).

3. Fase 2 — Pose Estimation

3.1 O que foi construído

Pose por pessoa (MediaPipe Pose), integrada ao mesmo pipeline da Fase 1 (mesmo stream, mesmo rastreador). **Três categorias de postura**, exatamente as do escopo mínimo do guia:

Categoria	Como é detectada	Severidade
Risco imediato (queda)	Orientação do torso (ombro-quadril) mais horizontal que vertical, em pixels	Crítica
Ergonomicamente inadequada (postura suspeita)	Cabeça projetada à frente do eixo dos ombros, em fração da altura do torso	Média
Aproximação a zona crítica	Centro da caixa da pessoa dentro do polígono de risco (configurável por câmera, desenhado no vídeo)	Alta

3.2 Métricas

- **PCK (Percentage of Correct Keypoints): não medido.** Exigiria um conjunto de imagens com a posição real das articulações marcada manualmente, que não existe no projeto e não há tempo hábil de construir. O MediaPipe Pose é um modelo de terceiros (não treinado pela equipe); sua acurácia publicada pelo fabricante é a referência válida para esse número.
- **Verificação da lógica de classificação:** 19 testes automatizados cobrem os três cenários acima com geometria de referência conhecida (pessoa em pé, pessoa caída, postura inclinada, dentro/fora da zona) — **100% de acerto** nesses casos controlados. Isso comprova que a lógica está correta; não substitui uma medição de recall em cenas reais, que depende de imagens rotuladas.

- **Teste ao vivo estruturado (23/09/2026, véspera da banca):** pessoa em pé (baseline, zero falso positivo), queda simulada com segurança, postura ruim proposital e entrada na zona marcada — as três categorias dispararam em cena real, cada uma com evidência de imagem salva (`/alerts/<id>/evidence`), lido do histórico real de alertas. Achado honesto: durante a queda, o rastreador trocou o ID da pessoa no meio do evento (limitação de continuidade do rastreamento, não da classificação de postura — ver limitação 8 no §10). O teste foi qualitativo, não uma contagem formal de tentativas-vs-acertos; um recall numérico com vídeo rotulado é o próximo passo (detalhe completo em `FASE2-POSE-ESTIMATION.md` §3).

3.3 Integração com a Fase 1

Roda no mesmo `CameraWorker`, mesmo quadro, sem recodificar a imagem — o rastreador de pessoas da Fase 1 também identifica de quem é cada pose, então "pessoa caída" já sai atribuído a "Pessoa 3", não a um alerta global sem dono.

4. Fase 3 — Notificação Inteligente

4.1 O que foi construído

- **Política de severidade explícita:** `critical` / `high` / `medium` / `low` / `info`, definida por regra (ex.: sem capacete = crítico; sem calçado = médio).
- **Dois canais de notificação, distintos:** 1. **Visual**, no dashboard (alertas agrupados por pessoa, com histórico e filtros). 2. **Sonoro**, na tela do operador — apita quando a portaria libera ou nega a entrada, perceptível por quem está fisicamente no local, sem depender de olhar a tela. Um terceiro canal (**Telegram**) está **planejado, não implementado** — depende da criação de um bot pela equipe (feito fora do sistema) e não entrou a tempo desta entrega, para não arriscar instabilidade às vésperas da apresentação.
- **Dashboard de gestão:** telas por perfil (operador só vê a câmera do setor; técnico configura tudo; supervisor administra contas e vê os gráficos de risco).
- **Modo portaria:** cada câmera pode exigir um conjunto próprio de EPIs; só libera a entrada se todos forem confirmados; qualquer incerteza (feature desligada, classe não suportada, fora do quadro) conta como reprovado. Veredito estabilizado por confirmação: nega em 3 quadros, libera em 8 — liberar por engano é o erro mais caro, por isso pesa mais.

4.2 Latência ponta a ponta (E2E), medida

Medida do início real da captura (violação já presente desde o primeiro quadro) até o alerta aparecer gravado no banco, 5 execuções contra o sistema rodando:

Execução	Latência	Alerta
1ª (partida fria — carregando o modelo)	5,97 s	crítico
2ª	1,22 s	alto
3ª	1,37 s	crítico
4ª	1,12 s	alto
5ª	1,18 s	alto

Com o sistema já aquecido: ~1,2 s de média. Meta do guia: $\leq 1s$ para crítico, $\leq 5s$ para alto. Ficamos **levemente acima da meta no crítico** (1,2 s vs. 1 s) e **dentro da meta no alto**, com folga.

5. Fase 4 — Validação em Ambiente Industrial

5.1 Conexão com câmeras reais

O sistema foi testado contra as câmeras RTSP reais da planta parceira. **Resultado: conectou em todas, e rodou em todas ao mesmo tempo, sem cair.** Achado observável: a latência da rede Wi-Fi até as câmeras prejudicou a detecção de forma perceptível (quadros mais espaçados, alguma degradação de qualidade), mas o sistema continuou funcionando, sem travar nem reiniciar sozinho.

5.2 Operação contínua

Testado **30 minutos seguidos, sem reinicialização manual** — funcionou de forma estável durante todo o período.

5.3 Confidencialidade das câmeras reais

As câmeras da planta real mostram trabalhadores e instalações reais — informação sensível conforme a política do guia (§2.5). **Usadas só em demonstração ao vivo, com autorização da empresa:** nenhuma imagem delas foi armazenada, publicada ou incluída em vídeo, slide ou neste repositório. Todo material público da entrega (vídeo institucional, demonstração gravada, slides) usa exclusivamente fontes públicas (Wikimedia Commons, licenças CC BY / domínio público, ver `docs/FIXTURES.md`) ou a própria webcam da equipe.

5.4 O que entrou nesta fase final

- **Protetor auricular** (7ª classe de EPI), fechando o mínimo de 5 EPIs exigido pelo escopo da Fase 1 (capacete, óculos, protetor auricular, calçado, colete).
- **Distinção uso correto/incorreto/ausente** (§2.1), fechando outro item do escopo mínimo da Fase 1.
- Testado ao vivo antes desta entrega: alertas `incorrect_helmet` e `missing_ear_protection` confirmados na tela, contra a webcam real.

6. Arquitetura

```
Câmera -> Captura (OpenCV) -> Visão computacional (ensemble YOLOv8 + rastreador + pose)
-> Regras e alertas (severidade, portaria) -> Tempo real (Socket.IO)
-> Dashboard (React) - por perfil: operador / técnico / supervisor
```

Sem componente de LLM/IA generativa nesta arquitetura — decisão deliberada desta fase final (ver §7).

7. Tecnologias

Categoria	Ferramentas
Linguagens	Python 3.11, TypeScript
Backend	Flask, Flask-SocketIO, SQLAlchemy, Alembic, SQLite
Frontend	React 18, Vite, Zustand
Visão computacional	YOLOv8 (Ultralytics), PyTorch + CUDA (GPU RTX), OpenCV, MediaPipe Pose
Qualidade	pytest (378 testes automatizados), Git/GitHub

Categoria	Ferramentas
Programação assistida por IA	Claude Code — usado ao longo de toda a implementação, sob direção e revisão da equipe (ver §8)
Planejamento	Trello

8. Política de uso de IA Generativa (declaração obrigatória)

A equipe usou **Claude Code** (assistente de programação por IA generativa, Anthropic) como ferramenta de apoio ao desenvolvimento, ao longo de todas as fases: geração de código sob direção da equipe, depuração, geração de testes automatizados, geração de documentação e apoio à criação de material de apresentação (slides, roteiro, vídeos). **Nenhum uso de imagem gerada por IA representando pessoa real** — todo material visual público usa fotos reais de bancos licenciados (Wikimedia Commons) ou a própria equipe. **Não foi usado LLM multimodal nesta entrega** (decisão de escopo: o guia da Challenge não exige, e a proximidade da apresentação ao vivo tornou o risco de uma dependência externa instável maior que o ganho — ver Sprint 3, currículo FIAP, para o uso de LLM multimodal nesse contexto separado).

9. Política de datasets e LGPD

- **SH17**: dataset público (CC BY-NC-SA 4.0, uso acadêmico), 8.099 imagens, usado como base de treino.
- **Vyra**: pesos prontos de terceiros (CC BY 4.0), atribuição devida ao autor ([Hexmon/vyra-yolo-ppe-detection](#)).
- **Imagens próprias**: a webcam da própria equipe foi usada nos testes e demonstrações. A equipe **não construiu um dataset próprio anotado** para treino — é uma limitação declarada; o modelo foi treinado só com dados públicos.
- **Pessoas reais**: as únicas pessoas identificáveis nas demonstrações públicas são a própria equipe (voluntários, consentimento implícito de quem participa do próprio projeto). As câmeras da planta real, com funcionários terceiros, foram usadas só ao vivo, nunca armazenadas ou publicadas (§5.3), em conformidade com a LGPD (Lei 13.709/2018) por minimização: o dado sensível simplesmente não é retido.

10. Limitações conhecidas (declaradas, não escondidas)

1. Métricas por classe medidas contra o conjunto de teste do **SH17** — do modelo isolado e também do ensemble completo (Vyra + SH17, ver [FASE1-DETECCAO-DE-EPIS.md](#) §3.1) — mas não em imagens da planta real, e não é o terreno de teste nativo do Vyra (medido lá, o ensemble sai pior em mAP na maioria das classes; achado honesto, explicado no relatório da Fase 1 — não é regressão real, é desalinhamento de gabarito).
2. PCK de pose não medido (falta dataset rotulado; ver §3.2).
3. Telegram (terceiro canal de notificação) planejado, não implementado.
4. Dataset de treino é 100% público; sem imagens próprias anotadas da equipe.
5. Alertas de "uso incorreto" são mais sensíveis a ruído do detector que os de "ausente" — qualquer detecção espúria fora de posição agora gera um alerta próprio, onde antes era descartada em silêncio. Risco aceito conscientemente nesta entrega.
6. `.env` de produção usa confirmação de 3 quadros para criar um alerta (não os 8 que chegaram a ser testados) — mantido assim deliberadamente por ser o valor já validado em uso, para não introduzir mudança de comportamento na véspera da apresentação.
7. O teste ao vivo de pose (queda/postura/zona, ver [FASE2-POSE-ESTIMATION.md](#) §3) foi qualitativo — confirma que cada categoria dispara em cena real, com evidência de imagem salva por alerta — mas não é uma contagem formal de tentativas-vs-acertos. Um recall numérico exige vídeo gravado e rotulado, roteiro cronometrado e árbitro

externo, que continua sendo o próximo passo.

8. O mesmo teste ao vivo expôs uma limitação do **rastreador de pessoas** durante quedas: a caixa delimitadora muda muito quando alguém cai, e o rastreador (IoU + distância de centro) por vezes perde continuidade e troca o ID da pessoa no meio do evento. A classificação de queda em si funcionou; o que fica exposto é a robustez do rastreamento de identidade nesse cenário específico.

11. Próximos passos

- Anotar um conjunto de teste próprio (ou usar o dataset nativo do Vyra) para medir a contribuição real do Vyra no ensemble — o teste contra o gabarito do SH17 (§3.1 da Fase 1) mede mal um modelo treinado em outro dataset.
- Fazer um teste de recall de pose formal (N tentativas conhecidas, vídeo gravado e rotulado) para um número de revocação em cena real, e não só a confirmação qualitativa já feita (ver limitação 7).
- Deixar o rastreador de pessoas mais robusto à mudança brusca de caixa durante quedas (limitação 8), para não trocar de identidade no meio de um evento crítico.
- Implementar o canal Telegram.
- Construir um dataset próprio anotado (fotos da planta ou de voluntários), para reduzir a dependência de dados só públicos.
- Reavaliar o uso de LLM multimodal como quarta camada de análise (Sprint 3), fora do caminho crítico do alerta.